Pertemuan 1:
Pengenalan Node.js

Node.js merupakan sebuah runtime environment yang dirancang untuk menjalankan JavaScript di sisi server. Dikembangkan oleh Ryan Dahl pada tahun 2009, Node.js memungkinkan para pengembang untuk menggunakan JavaScript—yang sebelumnya hanya digunakan di sisi klien (browser)—untuk membuat aplikasi server-side. Dengan demikian, Node.js mengubah paradigma pengembangan web dengan menyediakan platform yang efisien dan berskala tinggi untuk membangun aplikasi jaringan.

Keunggulan utama Node.js terletak pada pendekatannya yang non-blocking dan event-driven. Node.js menggunakan model I/O non-blocking, yang memungkinkan server untuk menangani banyak proses secara bersamaan tanpa menghambat kinerja. hal ini membuat Node.js ideal untuk aplikasi real-time, seperti aplikasi yang memerlukan layanan kolaborasi, permainan online, atau aliran data secara langsung. Dengan event-driven architecture, Node.js dapat merespon perubahan keadaan secara efisien dan meningkatkan responsivitas aplikasi.

Node.js juga dikenal dengan paket manajer dependensinya yang kuat, yaitu npm (Node Package Manager). Npm menyediakan akses ke ribuan paket atau modul yang dapat diintegrasikan ke dalam aplikasi Node.js dengan mudah. Hal ini mempercepat pengembangan, karena pengembang dapat memanfaatkan fungsionalitas yang sudah ada tanpa perlu menulis kode dari nol. Kemampuan untuk berbagi dan menggunakan paket-paket ini menjadikan Node.js sebagai salah satu platform pengembangan yang sangat produktif.

Dalam perkembangannya, Node.js telah menjadi salah satu teknologi yang sangat populer di dunia pengembangan web. Banyak perusahaan besar seperti Netflix, LinkedIn, dan Walmart menggunakan Node.js untuk membangun aplikasi mereka. Kecepatan eksekusi, fleksibilitas, dan kemampuan untuk menangani banyak koneksi secara bersamaan membuat Node.js menjadi pilihan yang sangat baik untuk aplikasi berskala besar dan real-time. Dengan terus berkembangnya ekosistemnya, Node.js tetap menjadi pemain kunci dalam dunia pengembangan web modern.

Node.js sering dipilih untuk pengembangan server-side dan aplikasi jaringan karena sejumlah alasan. Berikut adalah beberapa alasan mengapa seseorang mungkin memilih Node.js dan perbedaannya dengan penggunaan JavaScript di sisi klien (browser):

- **Non-blocking dan Event-Driven:** yang memungkinkan server untuk menangani banyak koneksi secara bersamaan tanpa menghambat kinerja. Ini membuatnya sangat cocok untuk aplikasi real-time yang memerlukan responsivitas tinggi.
- **Single Language:** dengan menggunakan JavaScript di kedua sisi (server dan klien), menciptakan keselarasan dalam pengembangan. Ini dapat mengurangi overhead dan mempermudah proses pengembangan.
- **NPM (Node Package Manager):** manajer paket yang sangat kuat. NPM memungkinkan pengembang untuk dengan mudah mengintegrasikan modul pihak ketiga ke dalam aplikasi mereka, menghemat waktu dan upaya pengembangan.
- **Skalabilitas:** memiliki kinerja tinggi dan skalabilitas yang baik, terutama untuk aplikasi real-time atau yang membutuhkan penanganan banyak koneksi secara bersamaan.

Instalasi

Ada beberapa cara untuk menginstal Node.js, tergantung pada sistem operasi yang kita gunakan.

Cara 1: Menggunakan installer resmi

Cara termudah untuk menginstal Node.js adalah menggunakan installer resmi yang disediakan oleh Node.js Foundation. Installer ini tersedia untuk berbagai sistem operasi, termasuk Windows, macOS, dan Linux. Untuk menginstal Node.js menggunakan installer resmi, ikuti langkah-langkah berikut:

- Buka situs web resmi Node.js pada tautan https://nodejs.org/en/download.
- Pilih versi Node.js yang sesuai dengan sistem operasi yang Kita gunakan.
- Klik tombol "Download".
- Jalankan file installer yang diunduh.
- Ikuti petunjuk pada layar untuk menyelesaikan instalasi.

Cara 2: Menggunakan manajer paket (Package Manager)

Jika kita menggunakan sistem operasi Linux, kita juga dapat menginstal Node.js menggunakan manajer paket. Manajer paket adalah alat yang memungkinkan kita untuk menginstal dan mengelola perangkat lunak dengan mudah.

Untuk menginstal Node.js menggunakan manajer paket, ikuti langkah-langkah berikut:

- Buka terminal.
- Ketik perintah berikut untuk menginstal Node.js:

```
sudo apt install nodejs
```

Cara 3: Menggunakan nvm

nvm adalah manajer versi Node.js yang memungkinkan kita untuk menginstal dan mengelola beberapa versi Node.js pada saat bersamaan. nvm tersedia untuk berbagai sistem operasi, termasuk Windows, macOS, dan Linux. Untuk menginstal Node.js menggunakan nvm, ikuti langkah-langkah berikut:

- Buka terminal.
- Ketik perintah berikut untuk menginstal nvm:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.1/install.sh | bash
```

- Restart terminal.
- Ketik perintah berikut untuk menginstal Node.js:

```
nvm install node
```

Pada umumnya, untuk menginstal Node.js, kita perlu mengunduh file installer atau menggunakan manajer paket. Setelah Node.js terinstal, kita dapat memeriksa versi Node.js yang terinstal dengan menggunakan perintah berikut:

```
node -v
```

IDE (Integrated Development Environment)

Adalah alat yang digunakan oleh pengembang perangkat lunak untuk menulis, mengedit, dan menjalankan kode. IDE yang umum digunakan untuk Node.js memiliki fitur-fitur berikut:

- Penyunting kode: IDE yang baik memiliki penyunting kode yang kuat dengan fitur-fitur seperti penyorotan sintaks, autocompletion, dan linting.
- Pembuat proyek: IDE yang baik memiliki pembuat proyek yang memudahkan pengembang untuk memulai proyek baru.

- Pengelola dependensi: IDE yang baik memiliki pengelola dependensi yang memungkinkan pengembang untuk mengelola dependensi proyek mereka.
- Penyelesaian masalah: IDE yang baik memiliki alat penyelesaian masalah yang dapat membantu pengembang menemukan dan memperbaiki kesalahan dalam kode mereka.

Berikut adalah beberapa IDE yang umum digunakan untuk Node.js:

- **Visual Studio Code**: IDE yang populer dan serbaguna yang dapat digunakan untuk mengembangkan berbagai jenis aplikasi, termasuk aplikasi Node.js karena memiliki fitur-fitur yang bervariasi.
- **WebStorm**: merupakan IDE yang dikembangkan oleh JetBrains yang khusus dirancang untuk pengembangan aplikasi web, termasuk aplikasi Node.js dan dukungan untuk framework web seperti React, Angular, dan Vue.
- **Atom**: IDE ini bersifat open source yang mudah disesuaikan dengan kebutuhan pengguna.
- **Sublime Text**: IDE yang ringan dan cepat dan dapat digunakan untuk mengembangkan berbagai jenis aplikasi.

Pada dasarnya, untuk memilih IDE yang tepat harus disesuaikan pada kebutuhan dan preferensi kita. Jika kita belum yakin dalam menentukan IDE, maka berikut ini ada beberapa faktor yang perlu dipertimbangkan:

- **Tingkat pengalaman:** Jika kita baru memulai dengan Node.js, kita mungkin ingin memilih IDE yang mudah digunakan.
- **Jenis proyek:** Jika Anda ingin mengembangkan proyek web, Anda mungkin ingin memilih IDE yang memiliki fitur-fitur khusus untuk pengembangan web.
- **Fitur yang kita butuhkan**: Pastikan IDE yang kita pilih, memiliki fitur-fitur yang kita butuhkan.

Kita pun bisa dengan mengunduh dan mencoba beberapa IDE yang berbeda untuk melihat mana yang paling sesuai dan nyaman.

Node.js Pertamamu

Setelah selesai proses instalasi, mari kita buat projek pertama, pada praktek ini, kita menggunakan Visual Studio Code (VS Code) sebagai langkah awal:

1. Buatlah folder dan buat file baru dengan nama 'app.js' (atau nama lain yang diinginkan), lalu tuliskan kode berikut:

```
const http = require('http') // menggunakan modul HTTP

const hostname = '127.0.0.1'// IP address server lokal
const port = 3000           // Port yang digunakan

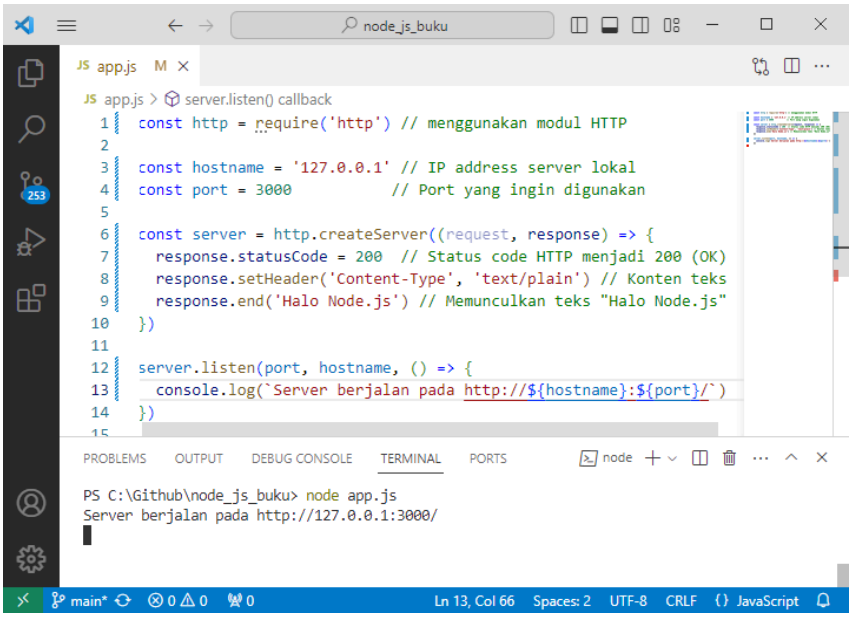
const server = http.createServer((request, response) => {
  response.statusCode = 200  // 200, menandakan OK
  response.setHeader('Content-Type', 'text/plain')
  response.end('Halo Node.js')
})

server.listen(port, hostname, () => {
  console.log(`Server berjalan pada http://${hostname}:${port}/`)
})
```

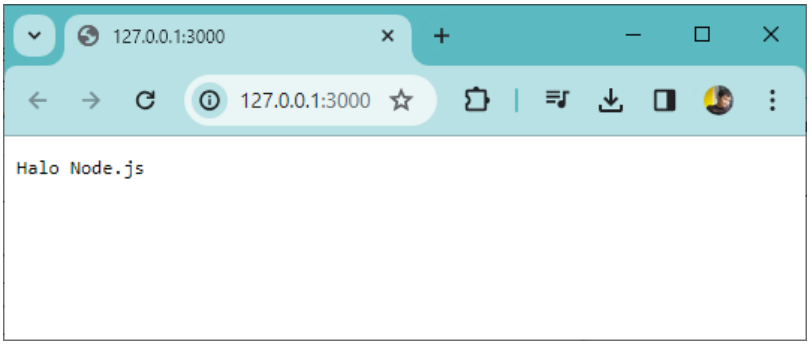
2. Jalankan kode dengan terminal, untuk memunculkannya, pilih terminal > new terminal, selanjutnya terminal akan muncul di bawah, kemudian ketik,

```
node app.js
```

Untuk membuka browser, tekan dan tahan tombol Ctrl pada keyboard, kemudian klik teks `http://127.0.0.1:3000/`.



Browser akan terbuka secara otomatis dan memunculkan hasil seperti dibawah ini,



3. Untuk mematikan server dengan menekan Ctrl dan C pada terminal.

Penjelasan kode:

1. `const http = require('http')` artinya memasukkan modul 'http' bawaan Node.js untuk membuat server web,
2. `const hostname = '127.0.0.1'` adalah alamat IP server lokal kita,
3. `const port = 3000` merupakan port yang akan digunakan oleh server,
4. `const server = http.createServer((req, res) => { ... })`membuat server web baru. Kalimat req, mengartikan **Request** dan res yang berarti **Result**.
5. `res.statusCode = 200` mendeteksi status code HTTP dengan nilai 200 yang berarti OK atau tak ada kendala.
6. `res.setHeader('Content-Type', 'text/plain')` menetapkan jenis konten yang akan dikirim ke browser sebagai teks biasa.
7. `res.end('Halo Node.js')` Mengirim pesan "Halo Node.js" ke browser dan mengakhiri respon.
8. `server.listen(port, hostname, () => { ... })` memulai server dan mendengarkan atau dalam kata lain "memantau" koneksi pada port yang ditentukan.

9. `console.log(`Server berjalan pada http://${hostname}:${port}/`)` mencetak pesan di terminal yang menunjukkan bahwa server sedang berjalan.

Perhatian:
Apabila kita melakukan modifikasi, harap simpan file atau dengan menekan Ctrl dan S, kemudian restart server dengan cara: tekan Ctrl dan C pada terminal, lalu ketik `node app.js` kembali dan tidak praktis.

Instalasi Nodemon

Kita yakin, bahwa melakukan restart server berkali-kali untuk melihat perubahan kode saat kita sedang bekerja akan merepotkan, tapi tenang, kita bisa membuat server agar melakukannya secara otomatis dengan menggunakan Nodemon, caranya:

1. Masuk ke terminal ketik,

```
npm init -y

npm install nodemon --save-dev
```

2. Buka file package.json, kemudian tambahkan kode berikut (yang ditebalkan) pada bagian scripts lalu simpan,

```
{
  ...
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js",
    ...
  },
  ...
}
```

3. Selanjutnya, matikan server (jika masih aktif) terlebih dahulu dengan Ctrl dan C pada terminal,
4. Jalankan server kembali dengan perintah `npm run dev.`

Perlu diingat! ketika nodemon sudah diinstall, maka setiap perubahan yang kita lakukan pada file app.js, cukup melakukan penyimpanan file atau Ctrl dan S, kemudian refresh browser saja untuk melihat perubahannya.

Web dengan Node.js

Pada bab sebelumnya, kita sudah diperkenalkan dengan modul HTTP atau Hypertext Transfer Protocol yang digunakan untuk berkomunikasi antara server dan klien. Protokol HTTP digunakan untuk mengirim dan menerima data dalam format teks. Dengan HTTP, kita dapat membuat aplikasi web, API, dan game berbasis browser. Pada Node.js, modul tersebut menyediakan berbagai fitur untuk menangani permintaan HTTP, seperti:

- Membaca permintaan HTTP
- Menulis respons HTTP
- Mendukung berbagai metode HTTP, seperti GET, POST, PUT, dan DELETE
- Mendukung berbagai format data, seperti text/plain, application/json, dan image/jpeg

Untuk membuat server HTTP dengan Node.js, kita dapat menggunakan fungsi createServer() dari modul HTTP.

HTML dengan Node.js

Pada Node.js, kode HTML dapat dikirimkan ke browser sebagai respons HTTP. Untuk melakukan hal ini, kita dapat menggunakan metode write() dari objek Response. Berikut adalah contoh kode untuk mengirimkan kode HTML sebagai respons HTTP, dalam contoh ini kita masih menggunakan file pada bahasan sebelumnya, yaitu app.js, mari kita ubah file tersebut:

```
const http = require('http') // modul HTTP

const hostname = '127.0.0.1' // IP address server lokal
const port = 3000            // Port yang digunakan

const server = http.createServer((request, response) => {
  response.writeHead(200, { 'Content-Type': 'text/html' })

  // Memunculkan teks "Halo Node.js" berupa HTML
  response.write('<html><body><h1>Halo Node.js</h1>
  </body></html>')
  response.end()
});

server.listen(port, hostname, () => {
  console.log(`Server berjalan pada http://${hostname}:${port}/`)
})
```

Kemudian simpan dan refresh browser. Jika kita klik kanan halaman browser dan pilih view page source, maka akan memunculkan tag HTML seperti ini,

```
<html>
  <body>
    <h1>Halo Node.js</h1>
  </body>
</html>
```

Selain itu, kita juga bisa menampilkan halaman HTML menggunakan Node.js, caranya:

1. Buat folder baru beri nama "html_pages" agar folder projek tertata rapi,
2. Pada folder html_pages, buatlah file HTML baru, beri nama "index.html",



3. Lanjut, isi dengan kode dibawah ini,

```
<html>
<head>
    <title>Halo</title>
</head>
<body>
    <h1>Halo Node.js</h1>

 <p>Hari ini cerah dan aku bersemangat</p>
</body>
</html>
```

4. Ubah app.js yang ditandai dengan teks tebal seperti dibawah ini,

```
const http = require('http') // modul HTTP
const fs = require('fs')     // modul file system

const hostname = '127.0.0.1' // IP address server lokal
const port = 3000            // Port yang ingin digunakan

const server = http.createServer((request, response) => {
  fs.readFile("./html_pages/index.html", (err, data) => {
    if (err) { // error
      response.writeHead(404, { 'Content-Type': 'text/html' })
      return response.end("<html><body><h1>404 Not Found</h1></body></html>")
    }

    // index.html
    response.writeHead(200, { 'Content-Type': 'text/html' })
    response.write(data)
    return response.end()
  })
})

server.listen(port, hostname, () => {
  console.log(`Server berjalan pada http://${hostname}:${port}/`)
})
```

5. Refresh browser sehingga memunculkan halaman HTML seperti dibawah,



Penjelasan kode:

Kode	Keterangan
`const fs = require('fs')`	Merupakan modul bawaan yang disediakan oleh Node.js untuk menggunakan File System (FS)
`fs.readFile("./html_page/index.html", (err, data) => { ... }`	File system membaca halaman berformat HTML
`if (err) { response.writeHead(404, {...}) return response.end(...) }`	Jika ada kegagalan saat membaca halaman HTML, maka pesan kesalahan dimunculkan dengan memberikan status 404 dan menampilkan HTML berupa "404 Not Found"
`response.writeHead(200, { 'Content-Type': 'text/html' }) response.write(data) return response.end()`	halaman HTML berhasil dimunculkan melalui browser dengan mengirimkan Status 200

URL (Uniform Resource Locator)

Pada contoh sebelumnya kita berhasil menampilkan satu halaman HTML saja, lalu bagaimana jika kita ingin memunculkan beberapa halaman HTML?. Untuk perihal ini Node.js menyediakan fitur bawaan untuk memproses URL dengan menyediakan beberapa fungsi, seperti:

1. Menerima dan memproses permintaan HTTP,
2. Membuat respons HTTP,
3. Melakukan operasi jaringan, seperti permintaan HTTP atau permintaan DNS.

Contoh kode untuk mengurai URL:

```
const parsedUrl =
new URL('https://contoh.com/web/sumber?nama=andi&usia=28')

console.log(parsedUrl)
```

Dari kode diatas akan menghasilkan keluaran seperti dibawah ini,

```
URL {
  href: 'https://contoh.com/web/sumber?nama=andi&usia=28',
  origin: 'https://contoh.com',
  protocol: 'https:',
```

```
  username: '',
  password: '',
  host: 'contoh.com',
  hostname: 'contoh.com',
  port: '',
  pathname: '/web/sumber',
  search: '?nama=andi&usia=28',
  searchParams: URLSearchParams { 'nama' => 'andi', 'usia' => '28' },
  hash: ''
}
```

Baiklah, mari kita lanjutkan pembahasan URL tersebut dengan halaman-halaman HTML yang akan kita buat. Kita akan membuat skenario bahwa akan ditambahkannya satu halaman HTML, berupa "profil.html" yang saling berkaitan dengan halaman "index.html":

1. Buatlah halaman HTML baru pada folder html_pages, beri nama "profil.html" dengan kode,

```
<html>
<head>
    <title>Profil</title>
</head>
<body>
    <p>
      <a href="/">Index</a>
      <a href="profil">Profil</a>
      <a href="kontak">Kontak</a>
    </p>

    <h1>Andrea Hirata Seman Said Harun</h1>
    <p>Novelis Indonesia yang berasal dari Pulau Belitung, provinsi Bangka Belitung.
Novel pertamanya adalah Laskar Pelangi yang menghasilkan tiga sekuel</p>
</body>
</html>
```

2. Tambahkan kode berikut, pada "index.html" seperti,

```
...
<p>
   <a href="/">Index</a>
   <a href="profil">Profil</a>
   <a href="kontak">Kontak</a>
</p>
<h1>Halo Node.js</h1>
...
```

3. Modifikasi app.js seperti tertera dibawah ini,

```
const http = require('http') // menggunakan modul HTTP
const fs = require('fs')     // menggunakan modul file system

const hostname = '127.0.0.1' // IP address server lokal
const port = 3000            // Port yang ingin digunakan

renderHTML = (path, response) => {
  fs.readFile(path, (err, data) => {
    if (err) {
      response.writeHead(404, { 'Content-Type': 'text/html' });
      return response.end("<html><body><h1>404 Not Found</h1></body></html>")
```

```
    }

    response.writeHead(200, { 'Content-Type': 'text/html' })
    response.write(data)
    return response.end()
  });
}

const server = http.createServer((request, response) => {
  const url = request.url

  if(url ==='/') {
    renderHTML('./html_pages/index.html', response)
  }else if (url === '/profil') {
    renderHTML('./html_pages/profil.html', response)
  }
});

server.listen(port, hostname, () => {
  console.log(`Server berjalan pada http://${hostname}:${port}/`);
})
```

4. Selesai, refresh browser dan klik navigasi untuk berpindah antar halaman HTML

HTTP Request Methods

Adalah bagian dari protokol HTTP yang digunakan untuk menentukan jenis operasi yang ingin dilakukan, HTTP methods memiliki delapan metode, yaitu:

- GET, digunakan untuk mengambil sumber daya, biasanya digunakan untuk mengambil halaman web, data, atau informasi lain dari server.
- POST, untuk membuat sumber daya baru, misalnya: untuk mengirimkan data ke server, seperti data pada HTML form atau API.
- PUT, memperbarui data atau informasi yang ada di server.
- DELETE, menghapus data atau informasi dari server.
- HEAD, mengambil informasi header dari sumber daya. Metode ini biasanya digunakan untuk memeriksa apakah sumber daya ada tanpa perlu mengambil ulang data dari server.
- OPTIONS, mendapatkan informasi tentang sumber daya. Metode ini biasanya digunakan untuk memeriksa apakah sumber daya dapat diakses atau tidak.
- TRACE, mengembalikan pesan yang dikirimkan oleh klien. Metode ini biasanya digunakan untuk debugging.

Guna mempermudah proses Request, kita memerlukan Modul "express" yang perlu kita install. Buka Terminal lalu ketik,

```
npm install express --save
```

Lanjut, kita buat beberapa contoh HTTP Methods dengan skenario pengiriman dan penerimaan data melalui HTML form seperti dibawah ini:

1. Dalam folder html_pages, tambahkan file HTML berupa "kontak.html" dengan kode berikut,

```
<html>
<head>
   <title>Kontak</title>
</head>
```

```
<body>
   <p>
      <a href="/">Index</a>
      <a href="profil">Profil</a>
      <a href="kontak">Kontak</a>
   </p>

   <h1>Kontak Penulis</h1>

   <form method="POST" action="/hasil">
      <p>Nama: <input type="text" name="nama"></p>
      <p>Subjek:
         <input type="radio" name="subjek" value="Pembicara"> Pembicara
         <input type="radio" name="subjek" value="Pelatihan"> Pelatihan
         <input type="radio" name="subjek" value="Talk Show"> Talk Show
      </p>
      <p><textarea name="pesan" cols="30" rows="5"></textarea></p>
      <p>
         <input type="reset" value="ULANGI">
         <input type="submit" value="KIRIM">
      </p>
   </form>
</body>
</html>
```

2. Mengingat bahwa kita sudah menggunakan Modul Express, maka perlu merubah seluruh file app.js seperti dibawah ini,

```
const express = require('express')
const path = require('path')

const app = express()
app.use(express.urlencoded({extended: true}))

app.get('/', (req, res) => {
  res.sendFile(path.join(__dirname,'./html_pages/index.html'))
})

app.get('/profil', (req, res) => {
  res.sendFile(path.join(__dirname,'./html_pages/profil.html'))
})

app.get('/kontak', (req, res) => {
  res.sendFile(path.join(__dirname,'./html_pages/kontak.html'))
})

app.post('/hasil', (req, res) => {
  data = req.body

  //kirimkan data sebagai cookie
  res.cookie('nama', data.nama)
  res.cookie('subjek', data.subjek)
  res.cookie('pesan', data.pesan)

  res.sendFile(path.join(__dirname,'./html_pages/hasil.html'))
})

app.listen(3000,() =>{
  console.log("Server berjalan pada http://localhost:3000")
})
```

3. Tambahkan pula halaman bernama "hasil.html" untuk menampilkan data yang dikirim melalui Node.js seperti,

```
<html>
<head>
   <title>Kontak diterima</title>
</head>
<body onload="show()">
   <p>Dari: <span id="nama"></span></p>
   <p>Subjek: <span id="subjek"></span></p>
   <p>Pesan: <span id="pesan"></span></p>
   <p><a href="/kontak">Kembali</a></p>

   <script language="javascript">
      function show() {
         let s_nama = document.getElementById("nama");
         let s_subjek = document.getElementById("subjek");
         let s_pesan = document.getElementById("pesan");

         s_nama.innerText = getCookie('nama');
         s_subjek.innerText = getCookie('subjek');
         s_pesan.innerText = getCookie('pesan');
      }

      function getCookie(k){
         let v = document.cookie.match('(^|;) ?'+k+'=([^;]*)(;|$)');
         return v?v[2].replaceAll("%20", ' '):null
      }
  </script>
</body>
</html>
```

Penjelasan kode:

Kode	Keterangan
File: kontak.html	
`<form method="POST" action="/hasil">`	Pada baris kode ini, Form mengatur hasil pengiriman data menggunakan metode POST yang akan diproses pada tautan '/hasil'
`Nama: <input type="text" name="nama">`	Akan membentuk textfield yang bernama "nama".
`<input type="radio" name="subjek" value="Pembicara"> Pembicara <input type="radio" name="subjek" value="Pelatihan"> Pelatihan <input type="radio" name="subjek" value="Talk Show"> Talk Show`	Objek ini disebut "Radio Button" yang hanya dapat dipilih salah satu dengan kesamaan nama, yaitu "subjek" namun memiliki "value" yang berbeda.
`<textarea name="pesan" cols="30" rows="5"></textarea>`	Jenis kotak masukkan yang sama dengan "text" namun memiliki bentuk yang panjang dan dapat memuat baris baru, sehingga cocok untuk menerima text yang panjang
`<input type="reset" value="ULANGI">`	Tipe "reset" berfungsi untuk mengosongkan field input
`<input type="submit" value="KIRIM">`	Berfungsi untuk mengirimkan nilai input

File: app.js	
`const http = require('http')`	Menggunakan modul HTTP
`const path = require('path')`	Menggunakan modul Path yang berfungsi membaca alur peletakkan file
`const express = require('express')`	Mengaktifkan modul express yang berguna dalam mempermudah proses Request
`const app = express()`	Mengaktifkan fitur express dan diekspresikan dalam variabel "app"
`app.use(express.urlencoded({extended: true}))`	Mengaktifkan fitur pembacaan URL, fitur ini diambil dari modul express

`app.get('/', (req, res) => {`
`  res.sendFile(path.join(__dirname, './html_pages/index.html'))`
`})`
	Menampilkan dengan metode GET halaman HTML statis bagi: index.html, profil.html dan kontak.html
	
	`res.sendFile` berfungsi menampilkan halaman HTML pada lokasi yang telah ditentukan oleh `path`

`app.post('/hasil', (req, res) => {`
`  data = req.body`
 	
`  res.cookie('nama', data.nama)`
`  res.cookie('subjek', data.subjek)`
`  res.cookie('pesan', data.pesan)`
 	
`res.sendFile(path.join(__dirname,'./html_pages/hasil.html'))`
`})`
	Memproses input yang diarahkan melalui Url '/hasil', kemudian menampung hasil data kedalam `data = req.body` dan meletakkannya sebagai cookie melalui `res.cookie(...)`.
	
	Pada `data.nama` bertujuan mengambil nilai "nama" yang dikirim melalui input HTML bernama "nama", begitu pula "subjek", dan "pesan", diakhiri membuka file hasil.html;

`app.listen(3000,() =>{`
`  console.log("Server berjalan pada http://localhost:3000")`
`})`
	Mengaktifkan server express

File: hasil.html	
`<body onload="show()">`	Menjalankan function `show()` saat tag body muncul pada browser

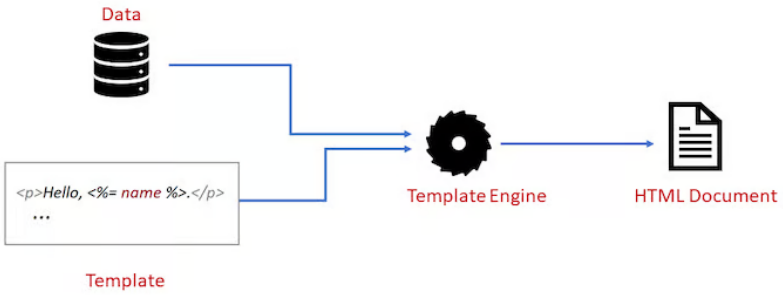
`<span id="nama"></span>`
`<span id="subjek"></span>`
`<span id="pesan"></span>`
	Merupakan tag yang digunakan sebagai tempat untuk menampilkan nama, subjek dan pesan

`function show() {`
`  let s_nama = document.getElementById("nama");`
`  ...`
 	
`  s_nama.innerText = getCookie('nama');`
`  ...`
`}`
	`let s_nama = document.getElementById("nama");` berfungsi untuk mengidentifikasi objek HTML yang memiliki id 'nama', begitu pula, subjek dan pesan, kemudian, meletakkan nilai dari cookie "nama", subjek dan pesan.
	
	`getCookie` merupakan function yang dibuat guna mengambil nilai dari cookie.

`function getCookie(k){`
`  let v = document.cookie.match('(^|;)?'+k+'=([^;]*)(;|$)');`
 	
`  return v?v[2].replaceAll("%20", ' '):null`
`}`
	Inilah function yang bertugas mengambil cookie, pada `document.cookie.match` melakukan penyaringan parameter yang berasal dari cookie, dan kemudian mengganti tanda `"%20"` menjadi spasi

Pertemuan 2:
Template Engine

Pada Bab sebelumnya kita melakukan pengiriman data bermetode POST antar halaman HTML, dan hal tersebut ternyata tidak disarankan oleh banyak pengembang sistem dikarenakan sulit saat melakukan penerapan dan tidak praktis. Node.js menyarankan kita untuk menggunakan template engine.



Konsep Template Engine
Sumber: https://blog.logrocket.com/how-to-use-ejs-template-node-js-application/

Template engine merupakan perangkat lunak yang berfungsi untuk membuat konten dinamis, ada banyak template engine yang tersedia untuk Node.js, masing-masing dengan kelebihan dan kekurangannya sendiri. Berikut adalah beberapa template engine Node.js yang populer:

- EJS (Embedded JavaScript),
- Handlebars,
- Pug (sebelumnya bernama Jade),
- Mustache,
- Dust.js,
- Nunjucks.

Pemilihan template engine yang tepat tergantung pada kebutuhan aplikasi kita. Jika mencari template engine yang sederhana dan mudah digunakan, EJS adalah pilihan yang baik. Jika Anda mencari template engine yang powerful dan fleksibel, Handlebars atau Pug adalah pilihan yang baik. Jika Anda mencari template engine yang cepat dan efisien, Mustache, Dust.js, atau Nunjucks adalah pilihan yang baik. Lalu bagaimana cara kita memilih template engine yang baik? Nah, berikut beberapa faktor hal yang perlu dipertimbangkan:

- Kesederhanaan: Seberapa mudah template engine tersebut digunakan?
- Kemampuan: Apakah template engine tersebut memiliki fitur yang kita butuhkan?
- Kinerja: Seberapa cepat template engine tersebut berjalan?
- Keterbacaan: Seberapa mudah template engine tersebut dibaca dan dipahami?
- Fleksibilitas: Seberapa fleksibel template engine tersebut?

Kita dapat mencoba beberapa template engine yang berbeda untuk melihat mana yang paling sesuai dengan kebutuhan.

EJS (Embedded JavaScript Templating)

EJS adalah mesin template JavaScript yang populer. EJS memungkinkan kita memasukkan kode JavaScript ke dalam template, yang kemudian digunakan untuk menghasilkan HTML. Panduan ini akan membahas cara menggunakan EJS pada aplikasi Node.js dimulai dengan dasar-dasar template engine dan menyiapkan EJS.

Pengaturan awal:

1. Buatlah folder standar EJS dan wajib bernama "views", selanjutnya tambahkan folder lain yang bertujuan agar file tertata rapi dengan memisahkan file utama dari komponen tambahan yang sering digunakan seperti, nav.ejs, footer.ejs dan head.ejs seperti tergambar dibawah,

2. Selanjutnya, instalasi. Pada terminal, ketik,

```
npm init -y

npm install express ejs
```

3. Buat kode, beri nama nav.ejs.

```
<nav>
    <a href="/">Index</a>
    <a href="/profil">Profil</a>
    <a href="/kontak">Kontak</a>
</nav>
<hr>
```

4. Masukkan kode pada footer.ejs berupa,

```
<hr>
<footer>
    <p>&copy; 2026 Modul Pembelajaran Node.js & EJS</p> </footer>
```

5. Lalu pada head.ejs,

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Aplikasi Web Node.js</title>
<style>
    body {
       font-family: Arial, sans-serif;
       margin: 20px;
       line-height: 1.6;
    }

    nav a {
       margin-right: 15px;
       text-decoration: none;
       color: #0066cc;
       font-weight: bold;
    }

    form div { margin-bottom: 12px; }
</style>
```

6. Pada halaman index.ejs berikan kode berikut.

```
<!DOCTYPE html>
<html lang="id">

<head>
    <%- include('../partials/head') %>
</head>
```

```
<body>
    <%- include('../partials/nav') %>

    <h1>Halo Node.js</h1>
    <p>Selamat datang di aplikasi web dinamis menggunakan Express dan EJS.</p>

    <%- include('../partials/footer') %>
</body>

</html>
```

7. Pada profil.ejs

```
<!DOCTYPE html>
<html lang="id">
<head>
    <%- include('../partials/head') %>
</head>
<body>
    <%- include('../partials/nav') %>

    <h1>Profil Penulis</h1>
    <h2>Andrea Hirata Seman Said Harun</h2>
    <p>Novelis Indonesia yang berasal dari Pulau Belitung, Provinsi Bangka Belitung.
Novel pertamanya adalah Laskar Pelangi.</p>

    <%- include('../partials/footer') %>
</body>
</html>
```

8. Pada file kontak.ejs

```
<!DOCTYPE html>
<html lang="id">
<head>
    <%- include('../partials/head') %>
</head>
<body>
    <%- include('../partials/nav') %>

    <h1>Kontak Penulis</h1>
    <form method="POST" action="/hasil">
        <div>
            <label>Nama:</label><br>
            <input type="text" name="nama" required>
        </div>
        <div>
            <label>Subjek:</label><br>
            <input type="radio" name="subjek" value="Pembicara" checked> Pembicara
            <input type="radio" name="subjek" value="Pelatihan"> Pelatihan
            <input type="radio" name="subjek" value="Talk Show"> Talk Show
        </div>
        <div>
            <label>Pesan:</label><br>
            <textarea name="pesan" cols="30" rows="5" required></textarea>
        </div>
        <div>
            <input type="reset" value="ULANGI">
            <input type="submit" value="KIRIM">
```

```
        </div>
    </form>

    <%- include('../partials/footer') %>
</body>
</html>
```

9. Untuk melihat hasil POST, buat file bernama hasil.ejs.

```
<!DOCTYPE html>
<html lang="id">
<head>
    <%- include('../partials/head') %>
</head>
<body>
    <%- include('../partials/nav') %>

    <h1>Pesan Berhasil Diterima!</h1>
    <p><strong>Dari:</strong> <%= data.nama %></p>
    <p><strong>Subjek:</strong> <%= data.subjek %></p>
    <p><strong>Pesan:</strong> <%= data.pesan %></p>
    <br>
    <a href="/kontak">Kirim Pesan Lain</a>

    <%- include('../partials/footer') %>
</body>
</html>
```

10. Ubah isi dari file app.js yang telah kita buat pada bahasan sebelumnya menjadi,

```
const express = require('express');
const path = require('path');
const app = express();

// Set EJS sebagai view engine
app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

// Middleware untuk memproses data dari HTML Form
app.use(express.urlencoded({ extended: true }));

// Routing GET
app.get('/', (req, res) => res.render('pages/index'));

app.get('/profil', (req, res) => res.render('pages/profil'));

app.get('/kontak', (req, res) => res.render('pages/kontak'));

// Routing POST untuk memproses form kontak
app.post('/hasil', (req, res) => {
    const dataForm = req.body;
    res.render('pages/hasil', { data: dataForm });
});

app.listen(3000, () => {
    console.log("Server berjalan pada http://localhost:3000");
});
```

11. Jalan server dengan `npm run dev` seperti biasa.

Pertemuan 3:
Google Firebase

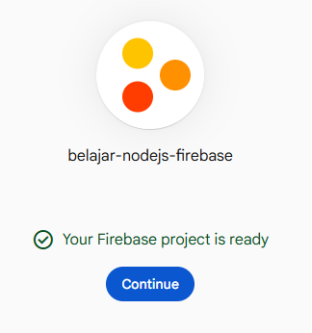
Pada pembahasan sebelumnya, data yang dikirim melalui form hanya ditampilkan secara sementara di halaman hasil dan hilang ketika halaman diperbarui atau server dimatikan. Untuk menyimpan data secara permanen (persistent), kita membutuhkan database.

Pada bagian ini, kita akan menggunakan Google Firebase Cloud Firestore, sebuah database NoSQL berbasis dokumen yang cepat, berskala tinggi, dan mudah terintegrasi dengan Node.js.

Konfigurasi Firebase Firestore

1. Membuat Proyek di Firebase Console

- Buka Firebase Console dan masuk menggunakan akun Google Anda.
- Klik **Create a new Firebase project**, masukkan nama proyek (misal: belajar-nodejs-firebase), lalu klik Continue dua kali.
- Perhatian!, hapus centang **Enable Google analytics for this project**, karena memerlukan pendaftaran dengan kartu kredit. klik Continue.
- Tunggu hingga muncul **Your Firebase project is ready**.



- Klik Continue untuk mengakhiri.

Bukalah Firestore pada panel sebelah kiri:

- Buka menu **Database and storage** > **Firestore Database**.
- Klik Create database, pilih Standard edition, lokasi server terdekat asia-southeast2 (Jakarta), lalu pilih Start in test mode untuk keperluan pembelajaran. Klik Create.

2. Mengunduh Service Account Key

Agar aplikasi Node.js kita mendapat izin untuk membaca dan menulis data ke Firebase, kita memerlukan Kunci Akses Admin (Private Key):

- Di Firebase Console, klik ikon roda gigi (Settings) di sebelah kiri atas > Service accounts.
- Pilih Node.js, lalu klik Generate new private key.
- Simpan file .json yang terunduh ke dalam direktori utama (root) proyek Node.js Anda dan ubah namanya menjadi serviceAccountKey.json.

Perhatian: Jangan pernah membagikan atau mengunggah file serviceAccountKey.json ke repositori publik (seperti GitHub) karena file ini berisi kredensial akses penuh ke database Anda.

Inisialisasi Firebase Admin SDK

Jalankan perintah berikut di terminal untuk menginstal SDK

```
npm install firebase-admin
```

Tunggu sampai proses selesai, perintah diatas membutuhkan waktu sedikit lama, sekitar 5 menit tergantung kecepatan internet kamu.

Buat file baru bernama firebase.js di folder proyek utama untuk mengelola koneksi database:

```
const { initializeApp, cert } = require('firebase-admin/app');
const { getFirestore } = require('firebase-admin/firestore');
const path = require('path');

// Lokasi path file serviceAccountKey.json
const serviceAccountPath = path.join(__dirname, 'serviceAccountKey.json');
const serviceAccount = require(serviceAccountPath);

// Inisialisasi Firebase App
initializeApp({
  credential: cert(serviceAccount)
});

// Inisialisasi Firestore
const db = getFirestore();

console.log("-> Berhasil terhubung ke Firebase Firestore!");

module.exports = db;
```

Operasi HTTP GET & POST dengan Firebase

Sekarang kita akan mengubah aplikasi kita agar setiap pesan yang dikirim dari form kontak langsung tersimpan ke Firestore, serta menambahkan halaman baru untuk melihat daftar semua pesan yang tersimpan di database.

1. Menambah Route POST (Menyimpan Data Ke Firestore)

Ubah handler app.post('/hasil') pada file app.js untuk meng-insert data ke koleksi Firestore bernama **pesan_kontak**:

```
Catatan: Tambah/ubah bagian teks tebal
```

```
const express = require('express');
const path = require('path');
const db = require('./firebase'); // Import Firestore

const app = express();

app.set('view engine', 'ejs');
app.set('views', path.join(__dirname, 'views'));

app.use(express.urlencoded({ extended: true }));

app.get('/', (req, res) => res.render('pages/index'));
app.get('/profil', (req, res) => res.render('pages/profil'));
app.get('/kontak', (req, res) => res.render('pages/kontak'));

// POST: Menyimpan  ke Firestore dengan async-await
app.post('/hasil', async (req, res) => {
    try {
```

```
        const { nama, subjek, pesan } = req.body;

        // Menyimpan data ke koleksi 'pesan_kontak'
        await db.collection('pesan_kontak').add({
            nama: nama,
            subjek: subjek,
            pesan: pesan,
            created_at: new Date()
        });

        // Tampilkan halaman hasil setelah simpan sukses
        res.render('pages/hasil', { data: req.body });
    } catch (error) {
        console.error("Gagal menyimpan data ke Firebase:", error);
        res.status(500).send("Terjadi kesalahan saat menyimpan data.");
    }
});
```

2. Menambah Route GET (Membaca Data Dari Firestore)

Tambahkan fitur baru di app.js untuk mengambil seluruh dokumen dari koleksi "pesan_kontak", menampilkan dan menghapusnya.

Tambahkan Route /daftar-pesan pada app.js.

```
Catatan: Menambah kode sebelum baris app.listen(3000...
```

```
// GET: Mengambil seluruh data dari 'pesan_kontak'
app.get('/daftar-pesan', async (req, res) => {
    try {
        // Mengambil snapshot dokumen yang diurutkan berdasarkan waktu pembuatan terbaru
        const snapshot = await db.collection('pesan_kontak').orderBy('created_at',
'desc').get();

        let pesanList = [];
        snapshot.forEach(doc => {
            pesanList.push({
                id: doc.id,
                ...doc.data()
            });
        });

        // Render halaman daftar pesan dengan mengirim data array
        res.render('pages/daftar-pesan', { pesanList: pesanList });
    } catch (error) {
        console.error("Gagal mengambil data dari Firebase:", error);
        res.status(500).send("Gagal memuat data pesan.");
    }
});

app.post('/hapus-pesan/:id', async (req, res) => {
    try {
        const docId = req.params.id;

        // Menghapus dokumen spesifik berdasarkan ID
        await db.collection('pesan_kontak').doc(docId).delete();

        console.log(`Pesan dengan ID ${docId} berhasil dihapus.`);

        // Redirect kembali ke halaman daftar pesan
```

```
        res.redirect('/daftar-pesan');
    } catch (error) {
        console.error("Gagal menghapus data dari Firebase:", error);
        res.status(500).send("Gagal menghapus pesan.");
    }
});

app.listen(3000, () => {
  ...
```

3. Membuat Halaman Tampilan Data.

Buat file baru bernama daftar-pesan.ejs di folder views/pages/.

```
<!DOCTYPE html>
<html lang="id">

<head>
    <%- include('../partials/head') %>
    <style>
        table {
           width: 100%;
           border-collapse: collapse;
           margin-top: 20px;
        }

        th,
        td {
           border: 1px solid #ddd;
           padding: 10px;
           text-align: left;
        }

        th {
            background-color: #f2f2f2;
        }
    </style>
</head>

<body>
    <%- include('../partials/nav') %>
    <h1>Daftar Pesan Masuk (Firebase Firestore)</h1>

    <% if (pesanList.length===0) { %>
       <p>Belum ada pesan yang masuk saat ini.</p>
       <% } else { %>
          <table>
             <thead>
                <tr>
                  <th>No</th>
                  <th>Nama</th>
                  <th>Subjek</th>
                  <th>Pesan</th>
                  <th>Waktu</th>
                  <th> </th>
                </tr>
              </thead>
              <tbody>
              <% pesanList.forEach((item, index)=> { %>
                <tr>
                    <td><%= index + 1 %></td>
```

```
                    <td><%= item.nama %></td>
                    <td><%= item.subjek %></td>
                    <td><%= item.pesan %></td>
                    <td><%= item.created_at ?
item.created_at.toDate().toLocaleString('id-ID') : '-' %></td>
                    <td>
                        <form action="/hapus-pesan/<%= item.id %>" method="POST"
onsubmit="return confirm('Apakah Anda yakin ingin menghapus pesan ini?');"> <button
type="submit" style="color: red;">Hapus</button> </form>
                    </td>
                </tr>
              <% }) %>
              </tbody>
           </table>
       <% } %>

       <%- include('../partials/footer') %>
</body>

</html>
```

4. Memperbarui Navigasi

Tambahkan tautan navigasi baru agar memudahkan perpindahan halaman.

nav.js *Tambah kode yang ditebalkan

```
<nav>
    <a href="/">Index</a>
    <a href="/profil">Profil</a>
    <a href="/kontak">Kontak</a>
    <a href="/daftar-pesan">Daftar Pesan</a>
</nav>
<hr>
```

Penjelasan:

Kode	Keterangan
`admin.initializeApp(...)`	Menginisialisasi koneksi SDK Firebase Admin menggunakan kunci autentikasi proyek (serviceAccountKey.json).
`db.collection('pesan_kontak')`	Menentukan koleksi target di Firestore yang digunakan untuk menyimpan atau mengambil data.
`.add({ ... })`	Metode pembawaan Firestore (HTTP POST) untuk menambah dokumen data baru dengan ID unik yang digenerate otomatis.
`.get()`	Metode pembawaan Firestore (HTTP GET) untuk membaca seluruh isi dokumen dari suatu koleksi.
`.orderBy('created_at', 'desc')`	Fungsi pengurutan data agar data pesan terbaru ditampilkan di urutan paling atas.
`doc.data()`	Mengambil objek isi data nyata dari satu entitas dokumen Firestore.